



CI1056 – ALGORITMOS E ESTRUTURA DE DADOS II

Profa. Rachel Reis
rachel@inf.ufpr.br



AULA DE HOJE

Quicksort

Material gentilmente disponibilizado pelo

Prof. André Vignatti

$\leq x$	x	$> x$
----------	-----	-------

- x é chamado de *pivô*

PARTIÇÃO DE VETOR

1. Problema Computacional

Partição de Vetor

Instância: (v, a, b, x) onde v é um vetor indexado por $[a..b]$ e x é um valor.

Resposta: modifica v de forma a garantir a existência de um índice $m \in [a - 1..b]$ tal que

$$v[i] \leq x, \forall i \in [a..m],$$

e

$$x < v[i], \forall i \in [m + 1..b].$$

Devolve este índice m .

ALGORITMO PARTICIONA

2. Algoritmo iterativo (não recursivo)

Particiona(v, a, b, x)

$m \leftarrow a - 1$

Para $i \leftarrow a$ to b

Se $v[i] \leq x$

$m \leftarrow m + 1$

Troca(v, m, i)

Devolva m

Ex1. Executar Particiona(v, 1, 6, v[6])

<i>i</i>	1	2	3	4	5	6
<i>v[i]</i>	6	10	13	5	8	3

Particiona(*v*, *a*, *b*, *x*)

$m \leftarrow a - 1$

Para $i \leftarrow a$ to b

Se $v[i] \leq x$

$m \leftarrow m + 1$

Troca(*v*, *m*, *i*)

Devolva *m*

Ex1. Executar Particiona(v, 1, 6, v[6])

	a ↓					b ↓
<i>i</i>	1	2	3	4	5	6
<i>v[i]</i>	6	10	13	5	8	3

↑
x

Particiona(v, 1, 6, 3)



Particiona(*v*, *a*, *b*, *x*)

$m \leftarrow a - 1$

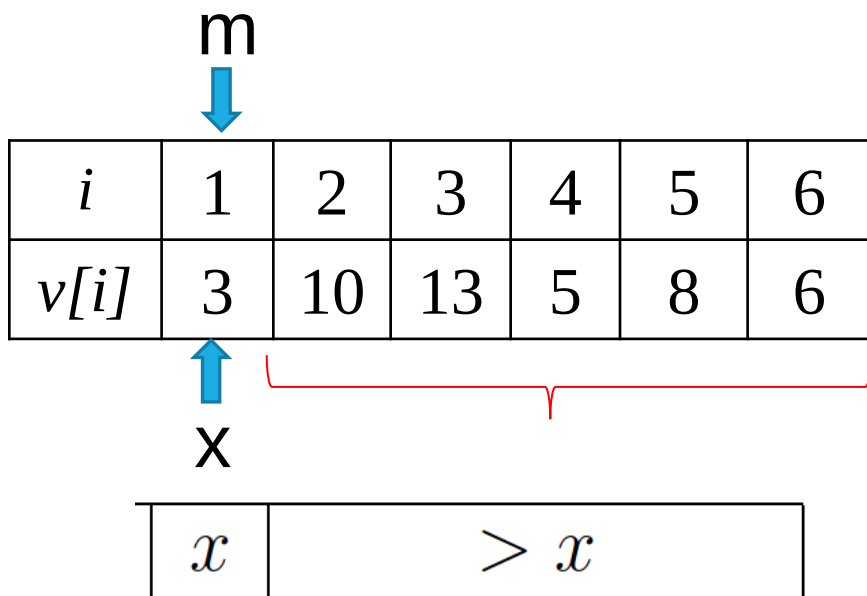
Para $i \leftarrow a$ to b

Se $v[i] \leq x$

$m \leftarrow m + 1$

Troca(v, m, i)

Devolva m



Particiona(v, a, b, x)

$m \leftarrow a - 1$

Para $i \leftarrow a$ to b

Se $v[i] \leq x$

$m \leftarrow m + 1$

Troca(v, m, i)

Devolva m

Ex2. Executar Particiona(v, 1, 6, v[6])

<i>i</i>	1	2	3	4	5	6
<i>v[i]</i>	3	10	13	5	8	6

Particiona(*v*, *a*, *b*, *x*)

$m \leftarrow a - 1$

Para $i \leftarrow a$ to b

Se $v[i] \leq x$

$m \leftarrow m + 1$

Troca(*v*, *m*, *i*)

Devolva *m*

Ex2. Executar Particiona(v, 1, 6, v[6])

	a					b
	↓					↓
<i>i</i>	1	2	3	4	5	6
<i>v[i]</i>	3	10	13	5	8	6
						↑
						x

Particiona(v, **1**, **6**, **6**)



Particiona(*v*, *a*, *b*, *x*)

$m \leftarrow a - 1$

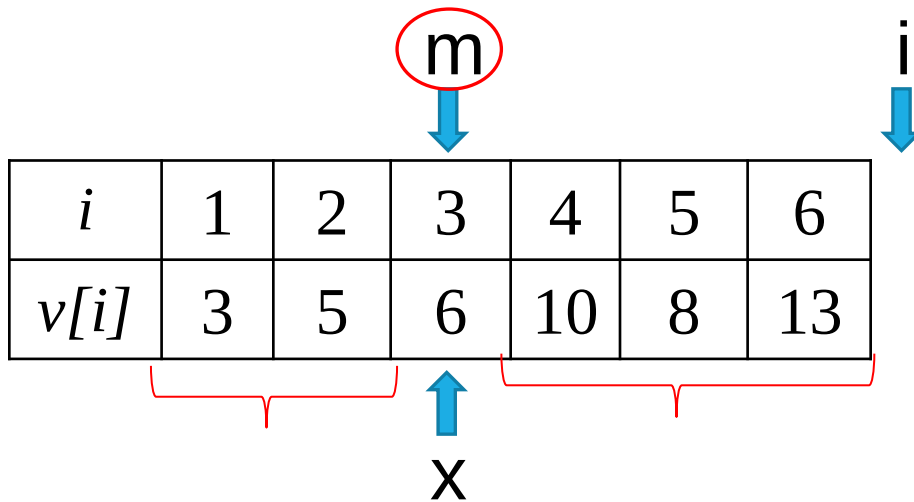
Para $i \leftarrow a$ to b

Se $v[i] \leq x$

$m \leftarrow m + 1$

Troca(v, m, i)

Devolva m



$\leq x$	x	$> x$
----------	-----	-------

Particiona(v, a, b, x)

$m \leftarrow a - 1$

Para $i \leftarrow a$ to b

Se $v[i] \leq x$

$m \leftarrow m + 1$

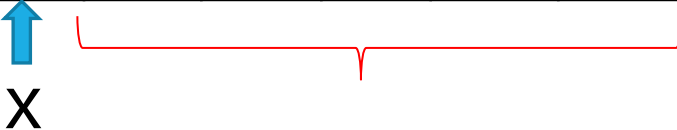
Troca(v, m, i)

 Devolva m

ALGORITMO PARTICIONA

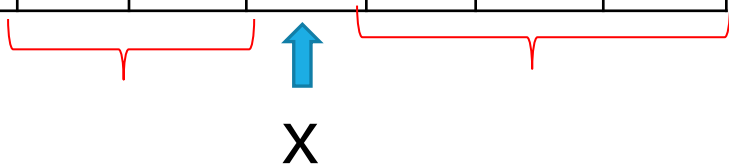
Ex1.

i	1	2	3	4	5	6
$v[i]$	3	10	13	5	8	6



Ex2.

i	1	2	3	4	5	6
$v[i]$	3	5	6	10	8	13



Importante:

Após a execução do Particiona, x é colocado na posição correta se o vetor estivesse ordenado.

PARTICIONA: ANÁLISE

- Seleção do recurso computacional:

$C_P(n)$ é o número de comparações com elementos do vetor em instâncias de tamanho n .

Particiona(v, a, b, x)

$m \leftarrow a - 1$

Para $i \leftarrow a$ to b

Se $v[i] \leq x$

$m \leftarrow m + 1$

Troca(v, m, i)

Devolva m

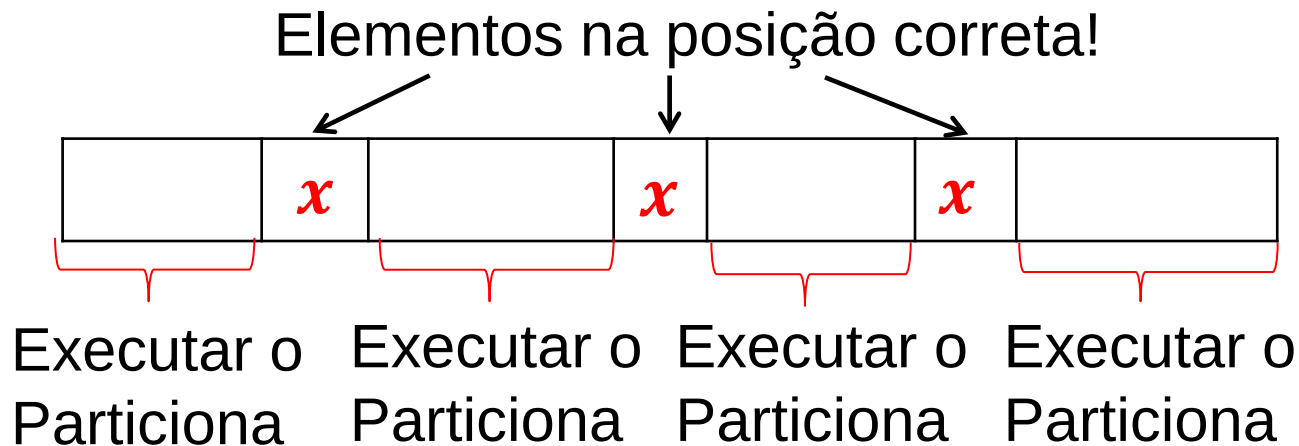
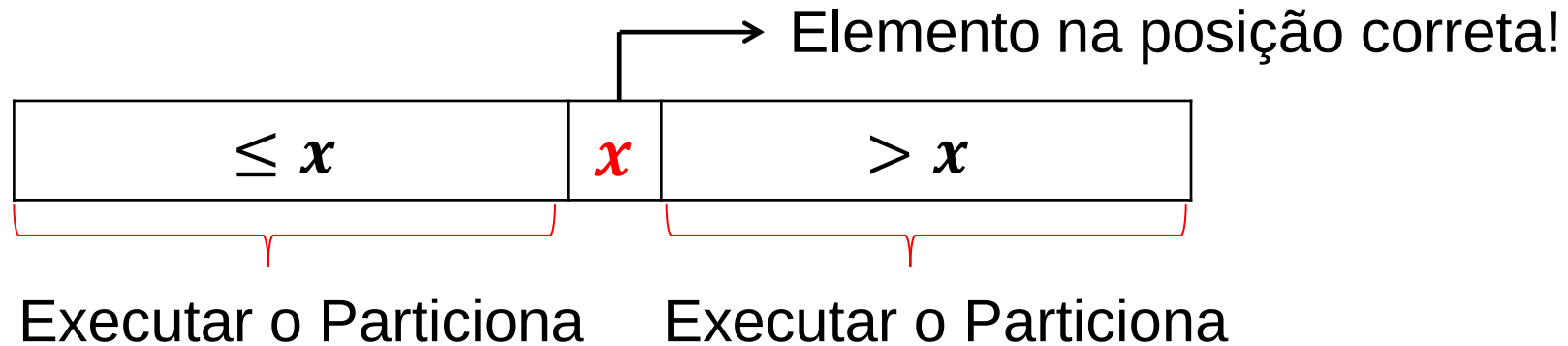
PARTICIONA: ANÁLISE

- Definição das linhas de interesse:
- As comparações são feitas sempre que o laço Para é executado.
- O laço é executado n vezes.
Logo, $C_P(n) = n$

Particiona(v, a, b, x)

$m \leftarrow a - 1$
Para $i \leftarrow a$ to b
 → Se $v[i] \leq x$
 $m \leftarrow m + 1$
 Troca(v, m, i)
Devolva m

SOBRE O ALGORITMO PARTICIONA...



QUICKSORT

- Algoritmo recursivo:

$$\text{Ordena}_Q(v, a, b)$$

Se $a \geq b$

 Devolva v

$m \leftarrow \text{Particiona}(v, a, b, v[b])$

$\text{Ordena}_Q(v, a, m - 1)$

$\text{Ordena}_Q(v, m + 1, b)$

É um algoritmo projetado usando a técnica “divisão e conquista”

Executar $Ordena_Q(v, 1, 6)$

i	1	2	3	4	5	6
$v[i]$	6	10	13	5	8	3

$main() \leftarrow O(v, 1, 6)$

$Ordena_Q(v, a, b)$

Se $a \geq b$

 Devolva v

$m \leftarrow \text{Particiona}(v, a, b, v[b])$

$Ordena_Q(v, a, m - 1)$

$Ordena_Q(v, m + 1, b)$

Executar $Ordena_Q(v, 1, 6)$

i	1	2	3	4	5	6
$v[i]$	3	5	6	8	10	13

$Ordena_Q(v, a, b)$

Se $a \geq b$

 Devolva v

$m \leftarrow \text{Particiona}(v, a, b, v[b])$

$Ordena_Q(v, a, m - 1)$

$Ordena_Q(v, m + 1, b)$

$\text{main}() \leftarrow O(v, 1, 6)$

$O(v, 1, 6) =$

$m \leftarrow 1$

~~$O(v, 1, 0) = v$~~

$O(v, 2, 6) =$

$m \leftarrow 3$

~~$O(v, 2, 2) = v$~~

$O(v, 4, 6) =$

$m \leftarrow 6$

$O(v, 4, 5) =$

$m \leftarrow 4$

~~$O(v, 4, 3) = v$~~

~~$O(v, 5, 5) = v$~~

 $\rightarrow$ Pivô

Executar $Ordena_Q(v, 1, 6)$

i	1	2	3	4	5	6
$v[i]$	3	5	6	8	10	13

main() $\leftarrow v$

$Ordena_Q(v, a, b)$

Se $a \geq b$

 Devolva v

$m \leftarrow \text{Particiona}(v, a, b, v[b])$

$Ordena_Q(v, a, m - 1)$

$Ordena_Q(v, m + 1, b)$

ANÁLISE

- Análise do algoritmo.

$C(n)$: número de comparações com elementos do vetor feitas pelo Quicksort.

ANÁLISE

Passo 1: Identificar as operações de interesse no algoritmo (que realizam comparações entre os elementos do vetor):

$$\text{Ordena}_Q(v, a, b)$$

Se $a \geq b$

Devolva v

→ $m \leftarrow \text{Particiona}(v, a, b, v[b])$

→ $\text{Ordena}_Q(v, a, m - 1)$

→ $\text{Ordena}_Q(v, m + 1, b)$

ANÁLISE

Passo 2: Dividir a análise em base e não base:

- Definição da não base:

$$C(n) = \begin{cases} 0, & \text{se } n \leq 1 \\ C(k) + C(n - k - 1) + C_P(n), & \text{se } n > 1 \end{cases}$$

$\text{Ordena}_Q(v, a, b)$

Se $a \geq b$

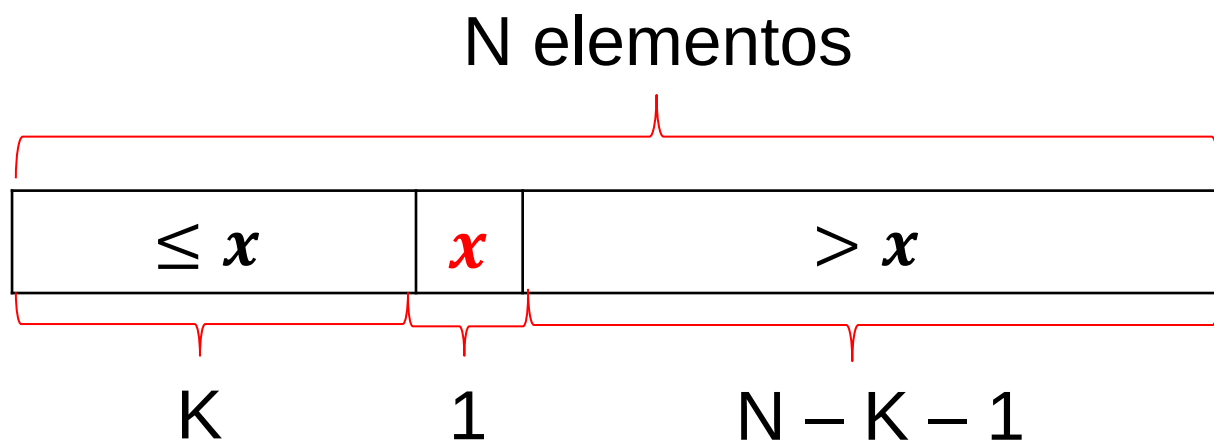
 Devolva v

$m \leftarrow \text{Particiona}(v, a, b, v[b])$

$\text{Ordena}_Q(v, a, m - 1)$

$\text{Ordena}_Q(v, m + 1, b)$

ANÁLISE



onde k e $n - k - 1$ são os tamanhos das partições obtidas.

ANÁLISE

Substituindo $C_P(n) = n$,

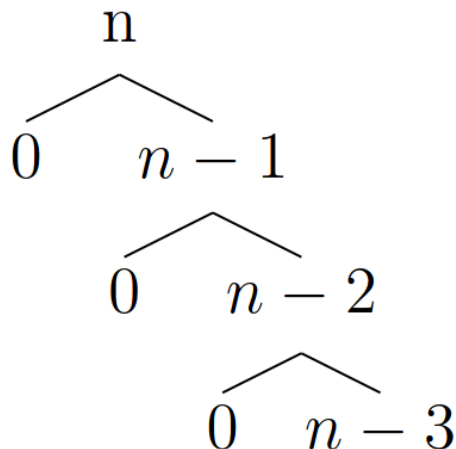
$$C(n) = \begin{cases} 0, & \text{se } n \leq 1 \\ C(k) + C(n - k - 1) + n, & \text{se } n > 1 \end{cases}$$

RACIOCINANDO...

O que vocês acham que seria um
melhor caso e um **pior caso**?

ANÁLISE: PIOR CASO

Um **caso ruim** é quando, por exemplo, o Particiona produz partições de tamanho 0 e $n - 1$ (vetor já ordenado).



Esse caso ruim é o **pior caso**?

- Sim, mas não vamos provar isso aqui



Análise de Algoritmos

$$C(n) = \begin{cases} 0, & \text{se } n \leq 1 \\ C(k) + C(n - k - 1) + n, & \text{se } n > 1 \end{cases}$$

ANÁLISE: PIOR CASO

Assim, a relação de recorrência fica:

$$C(n) = \begin{cases} 0, & \text{se } n \leq 1 \\ C(0) + C(n - 1) + n, & \text{se } n > 1 \end{cases}$$

ou

$$C(n) = \begin{cases} 0, & \text{se } n \leq 1 \\ C(n - 1) + n, & \text{se } n > 1 \end{cases}$$

Resolver a recorrência

$$\begin{aligned} C(n) &= C(n - 1) + n \\ &= C(n - 2) + n - 1 + n \\ &= C(n - 3) + n - 2 + n - 1 + n \\ &\vdots \\ &= C(n - \mathbf{i}) + (n - (\mathbf{i} - 1)) + (n - (\mathbf{i} - 2)) + \dots + (n) \end{aligned}$$

Quando pára?

$$\mathbf{n - i = 1}$$

$$\mathbf{i = n - 1}$$

$$C(n) = \begin{cases} 0, & \text{se } n \leq 1 \\ C(n - 1) + n, & \text{se } n > 1 \end{cases}$$

Resolver a recorrência

$$C(n) = C(n - 1) + n$$

$$= C(n - 2) + n - 1 + n$$

$$= C(n - 3) + n - 2 + n - 1 + n$$

⋮

$$= C(n - (\mathbf{n-1})) + (n - ((\mathbf{n-1}) - 1)) + (n - ((\mathbf{n-1}) - 2)) + \dots + (n)$$

Substituir



i = n - 1

Resolver a recorrência

$$\begin{aligned}C(n) &= C(n - 1) + n \\&= C(n - 2) + n - 1 + n \\&= C(n - 3) + n - 2 + n - 1 + n \\&\vdots \\&= C(n - (n-1)) + (n - ((n-1) - 1)) + (n - ((n-1) - 2)) + \dots + (n) \\&= C(1) + 2 + 3 + \dots + n \\&= 0 + 2 + 3 + \dots + n \\&= 2 + 3 + \dots + n \\&= \sum_{i=2}^n i = \frac{(n^2 + n - 2)}{2} = \frac{n^2}{2} \left(1 + \frac{1}{n} - \frac{2}{n^2}\right) \approx \frac{n^2}{2}\end{aligned}$$

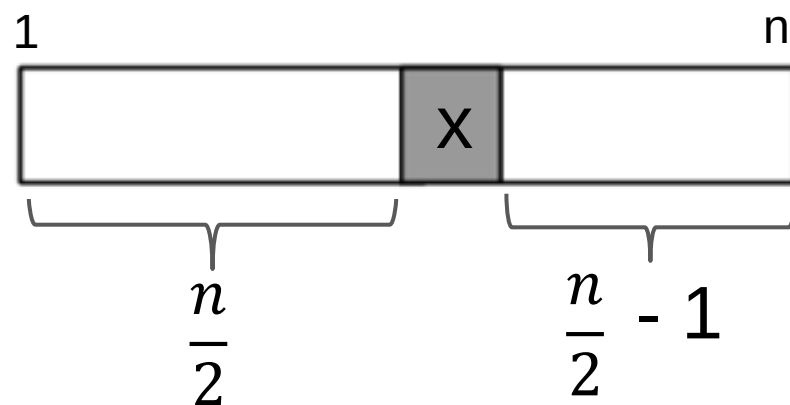
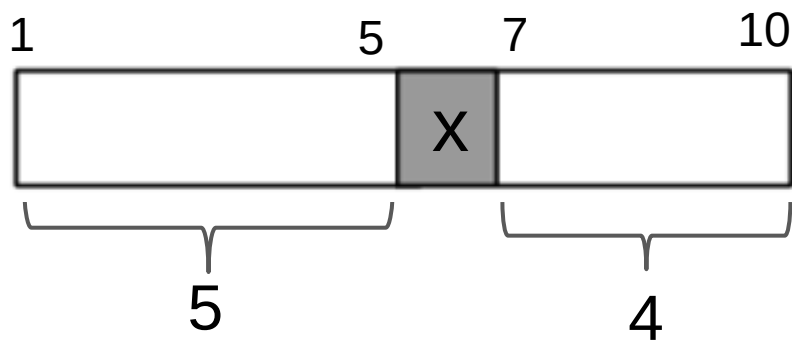
CONCLUSÃO – PIOR CASO

- O algoritmo **Quicksort**, no pior caso, faz o mesmo número de comparações que o algoritmo **SelectionSort**.
- O algoritmo **Quicksort**, no pior caso, faz o mesmo número de comparações que o algoritmo **InsertionSort** (no pior caso).

ANÁLISE: MELHOR CASO

Ocorre quando o algoritmo Particiona produz as duas partições com tamanho “igual”.

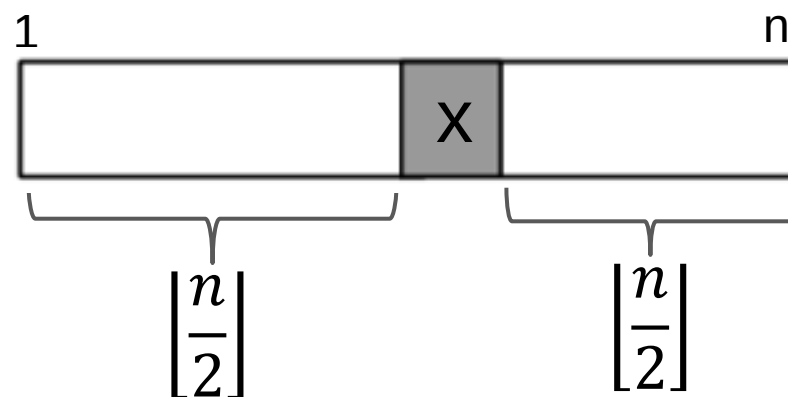
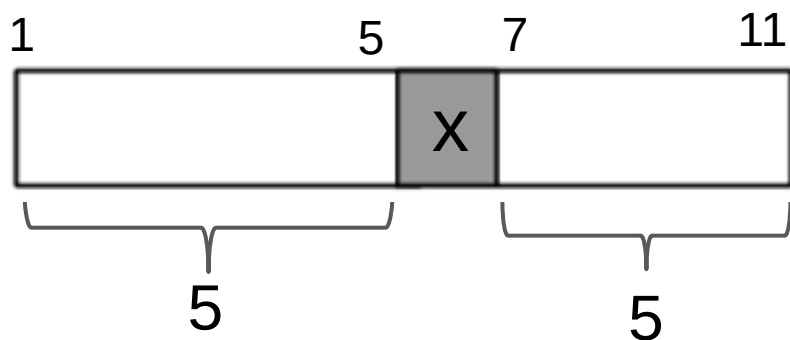
Caso n par:



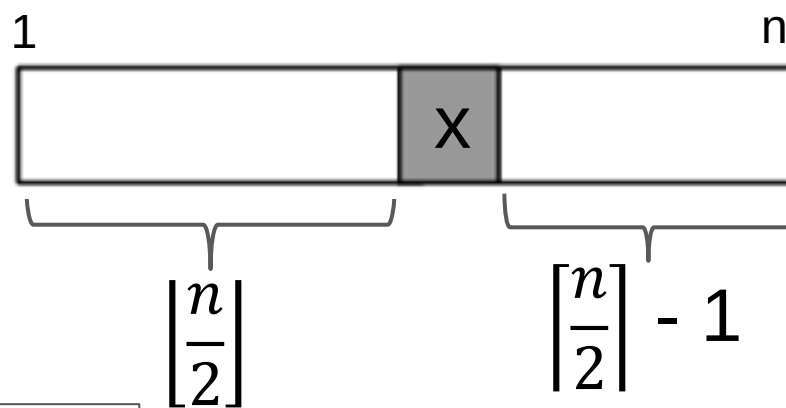
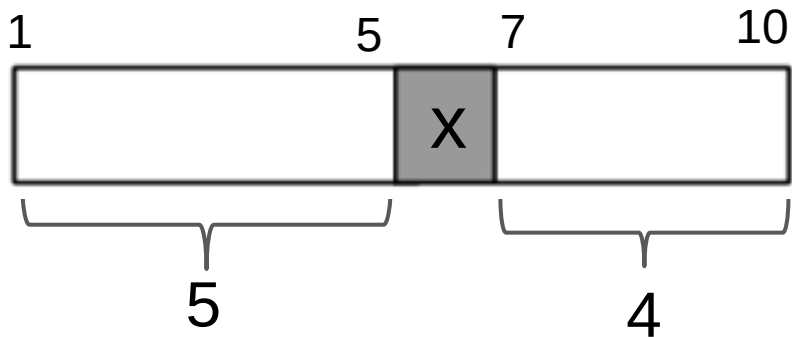
ANÁLISE: MELHOR CASO

Ocorre quando o algoritmo Particiona produz as duas partições com tamanho “igual”.

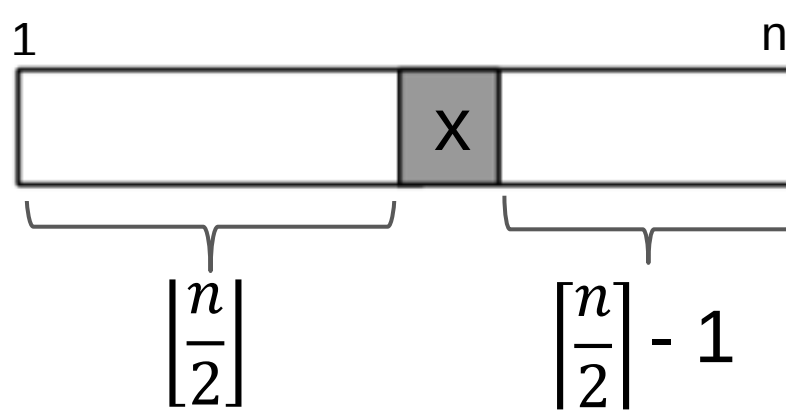
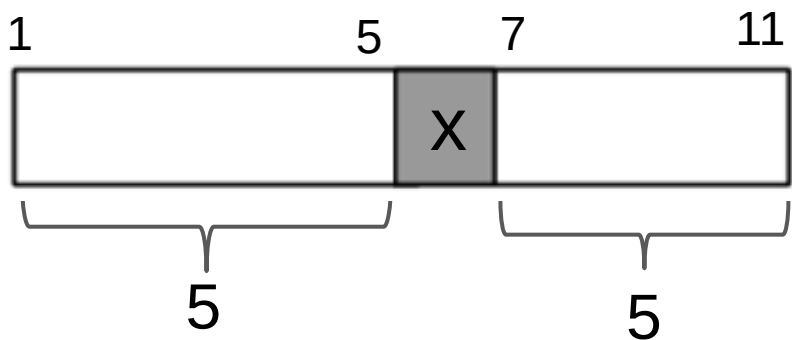
Caso n ímpar:



n par



n ímpar



ANÁLISE: MELHOR CASO

Ocorre quando o algoritmo Particiona produz as duas partições com tamanho “igual”.

Caso n par:

$\lfloor n/2 \rfloor$	x	$\lceil n/2 \rceil - 1$
-----------------------	----------	-------------------------

Caso n ímpar:

$\lfloor n/2 \rfloor$	x	$\lceil n/2 \rceil - 1$
-----------------------	----------	-------------------------

$$C(n) = \begin{cases} 0, & \text{se } n \leq 1 \\ C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil - 1) + n, & \text{se } n > 1 \end{cases}$$

ANÁLISE: MELHOR CASO

Teorema. *A relação de recorrência:*

$$C(n) = \begin{cases} 0, & \text{se } n \leq 1 \\ C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil - 1) + n, & \text{se } n > 1 \end{cases}$$

tem como solução $C(n) \approx n \log_2 n$.

Resolver essa recorrência é **difícil**  **Análise de Algoritmos**

Mas assumindo simplificações, podemos ter uma ideia da demonstração do teorema.

ANÁLISE: MELHOR CASO

Simplificação 1: Assumimos que n é potência de 2, ou seja, $n = 2^k$

Simplificação 2: $C(\lceil n/2 \rceil - 1)$ fazemos ser igual a $C(\lceil n/2 \rceil)$

$$C(n) = \begin{cases} 0, & \text{se } n \leq 1 \\ 2C(\frac{n}{2}) + n, & \text{se } n > 1 \end{cases}$$

Resolver a recorrência

$$C(n) = \begin{cases} 0, & \text{se } n \leq 1 \\ 2C\left(\frac{n}{2}\right) + n, & \text{se } n > 1 \end{cases}$$

$$\begin{aligned} C(n) &= 2C\left(\frac{n}{2}\right) + n \\ &= 2\left(2C\left(\frac{n}{2^2}\right) + \frac{n}{2}\right) + n \\ &\quad \vdots \\ &= 2^{\log_2 n} C\left(\frac{n}{2^{\log_2 n}}\right) + \log_2 n \cdot n \\ &= 2^{\log_2 n} C(1) + \log_2 n \cdot n \end{aligned}$$

$$C(n) = \mathbf{n \log_2 n}$$

QUICKSORT: ALGUMAS OBSERVAÇÕES

Melhor caso

$$C(n) = n \log_2 n$$

Pior caso

$$C(n) = \frac{n^2}{2}$$

Na teoria: Seu pior caso é aproximadamente igual aos dos algoritmos mais simples de ordenação.

Na prática: O Quicksort é um dos mais eficientes algoritmos de ordenação